

Topic 2: K-Nearest Neighbors (KNN)

0.1 K-Nearest Neighbors (KNN)

0.1.1 The Simplest Idea in Machine Learning

K-Nearest Neighbors may be the most intuitive algorithm in this whole book, because you already use it in daily life.

Intuition & Analogy

You move to a new city and want to know whether a neighbourhood is quiet or lively. What do you do? You ask the people who live *closest* to the house you are considering. If 4 of your 5 nearest neighbours say “quiet”, you conclude the area is quiet. That is the entire KNN algorithm: *to classify something new, look at the k most similar known examples and take a majority vote.*

Definition: K-Nearest Neighbors

KNN classifies a new data point by finding the k training examples closest to it (its “nearest neighbours”) and assigning the class that appears most often among them. It performs no learning in advance — it simply memorises the training data and defers all work until prediction time. For this reason it is called a **lazy learner** and an **instance-based** method.

0.1.2 Step 1: Measuring “Closeness” — Distance Metrics

To find the nearest neighbours we must define what “near” means. The most common choice is the **Euclidean distance** — the ordinary straight-line, ruler-on-a-map distance. For two points $A = (a_1, a_2, \dots, a_n)$ and $B = (b_1, b_2, \dots, b_n)$:

$$d(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2} \quad (1)$$

With two features this is just the Pythagorean theorem from school geometry.

An alternative is the **Manhattan distance**, which measures distance as if you were walking along the streets of a grid-shaped city (no diagonal shortcuts):

$$d(A, B) = |a_1 - b_1| + |a_2 - b_2| + \dots + |a_n - b_n| \quad (2)$$

0.1.3 Step 2: The Algorithm, Step by Step

To classify a new point:

1. Compute the distance from the new point to *every* training point.

2. Sort the training points by distance and keep the k closest ones.
3. Count the class labels among these k neighbours.
4. Predict the class with the most votes (the majority class).

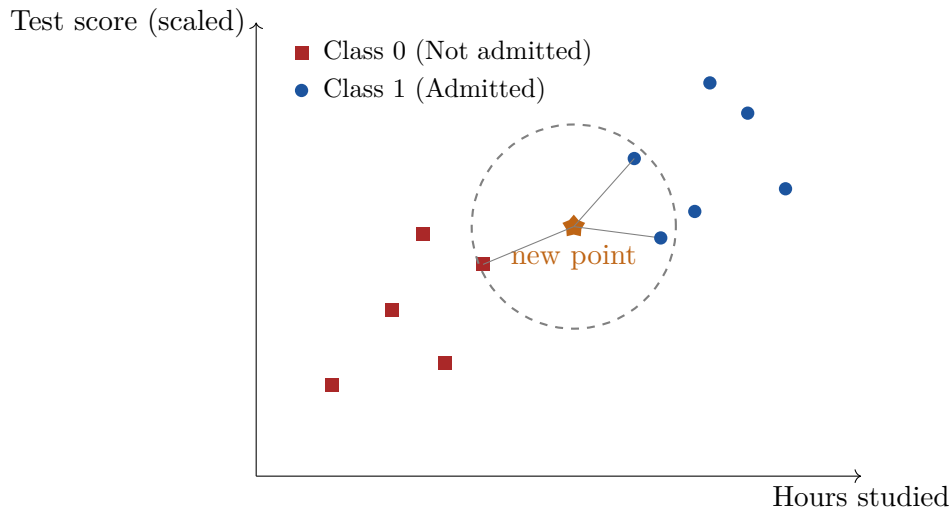


Figure 1: KNN with $k = 3$. The dashed circle encloses the three nearest neighbours of the new point: two blue circles (class 1) and one red square (class 0). The vote is 2–1 for class 1, so the new point is predicted *Admitted*.

0.1.4 Step 3: Choosing k — The Crowd-Size Dilemma

The value of k controls how many neighbours get a vote, and it dramatically changes behaviour:

- **Small k (e.g. $k = 1$):** The prediction depends entirely on the single closest point. The decision boundary becomes jagged and nervous, chasing every noisy point. This is **overfitting** — like judging a whole neighbourhood by one eccentric resident.
- **Large k (e.g. $k = \text{all points}$):** Every prediction becomes the overall majority class of the entire dataset, ignoring local patterns. This is **underfitting** — like polling the whole country to decide whether one street is quiet.
- **The sweet spot** is found by trying several values of k and measuring performance on validation data (*cross-validation*). Odd values of k are preferred in binary problems to avoid tied votes. A practical alternative used in the companion notebook is the **elbow method**: plot the test error rate against k and pick the value where the error settles down.

0.1.5 The Critical Importance of Feature Scaling

Suppose one feature is “hours studied” (range 0–10) and another is “annual family income” (range 0–100,000). In the Euclidean distance formula, a difference of 5,000 in income utterly drowns a difference of 5 hours of study. The distance — and therefore the entire algorithm — would be dictated by income alone.

The fix is to **scale** all features to comparable ranges before running KNN, for example with standardisation:

$$x_{\text{scaled}} = \frac{x - \mu}{s} \quad (3)$$

where μ is the feature's mean and s its standard deviation. After scaling, every feature has mean 0 and standard deviation 1, and each gets a fair say in the distance. *For KNN, scaling is not optional — it is essential.*

Worked Example: Classifying a New Applicant with KNN ($k = 3$)

Using the bootcamp dataset from Topic 1, classify a new applicant with 5.5 hours studied and a test score of 63. For simplicity we compute raw Euclidean distances (in practice we would scale first).

Applicant	(hours, score)	Distance calculation	Distance	Label
A	(2, 45)	$\sqrt{(5.5 - 2)^2 + (63 - 45)^2}$	18.34	0
B	(4, 55)	$\sqrt{(5.5 - 4)^2 + (63 - 55)^2}$	8.14	0
C	(5, 60)	$\sqrt{(5.5 - 5)^2 + (63 - 60)^2}$	3.04	0
D	(6, 62)	$\sqrt{(5.5 - 6)^2 + (63 - 62)^2}$	1.12	1
E	(8, 70)	$\sqrt{(5.5 - 8)^2 + (63 - 70)^2}$	7.43	1
F	(10, 85)	$\sqrt{(5.5 - 10)^2 + (63 - 85)^2}$	22.46	1

The three smallest distances belong to D (1.12, label 1), C (3.04, label 0), and E (7.43, label 1). The vote is **2–1 in favour of class 1**, so the applicant is predicted **Admitted**. Note how the answer could change with $k = 1$ (still 1, via D) or with a different scaling of the features — KNN's verdict depends on both choices.

Hands-On Python Notebook

The full Python implementation for this topic lives in the companion notebook `02_K_Nearest_Neighbors.ipynb`. It covers (i) KNN on the mini bootcamp dataset with a cross-validated search over k , and (ii) a complete real-data workflow on an anonymised “classified” dataset (`Classified_Data`): standardising the variables with `StandardScaler`, train/test split, fitting `KNeighborsClassifier`, evaluating with a confusion matrix and classification report, and choosing k with the elbow method.

0.1.6 Assumptions and Requirements

- Points that are close together in feature space tend to share the same class (the “smoothness” assumption). If similarity in features does not imply similarity in class, KNN fails.
- Features must be numeric and *scaled*; categorical features need encoding first.
- Works best with a modest number of features. In very high dimensions all points become roughly equally far apart — the **curse of dimensionality** — and “nearest” loses its meaning.

0.1.7 Advantages and Disadvantages

Common Mistakes

- **Forgetting to scale features.** This is the single most common KNN error, and it silently produces garbage.
- **Choosing k by guesswork** instead of cross-validation.
- **Using an even k in binary classification**, inviting tied votes.

Table 1: Strengths and weaknesses of KNN.

Advantages	Disadvantages
Extremely simple to understand and explain	Prediction is slow on large datasets (must scan all points)
No training phase at all	Must store the entire training set in memory
Naturally handles multiclass problems	Very sensitive to feature scaling and irrelevant features
Can learn arbitrarily curved decision boundaries	Suffers badly in high-dimensional spaces
New data can be added without retraining	Sensitive to noisy points when k is small

- **Applying KNN to huge datasets** without approximate-neighbour tricks; prediction time grows with dataset size.
- **Keeping irrelevant features.** Every feature affects the distance, so useless features add pure noise.

Key Points

- KNN classifies by a majority vote among the k nearest training points.
- It memorises data instead of learning parameters — a lazy learner.
- Small k overfits; large k underfits; choose k by cross-validation (or the elbow method).
- Feature scaling is mandatory because distances mix all features.